Like all other validations, `validates_with` takes the `:if`, `:unless` and `:on` options. If you pass any other options, it will send those options to the validator class as `options`:

```
class GoodnessValidator < ActiveModel::Validator
  def validate(record)
    if options[:fields].any?{|field| record.send(field) == "Evil" }
      record.errors[:base] << "This person is evil"
    end
  end
end

class Person < ActiveRecord::Base
  validates_with GoodnessValidator, fields: [:first_name, :last_name]
end
```

Note that the validator will be initialized *only once* for the whole application life cycle, and not on each validation run, so be careful about using instance variables inside it.

If your validator is complex enough that you want instance variables, you can easily use a plain old Ruby object instead:

```
class Person < ActiveRecord::Base
  validate do |person|
    GoodnessValidator.new(person).validate
  end
end

class GoodnessValidator
  def initialize(person)
    @person = person
  end

  def validate
    if some_complex_condition_involving_ivars_and_private_methods?
      @person.errors[:base] << "This person is evil"
    end
  end

  # ...
end
```

2.13 `validates_each`

This helper validates attributes against a block. It doesn't have a predefined validation function. You should create one using a block, and every attribute passed to `validates_each` will be tested against it. In the following example, we don't want names and surnames to begin with lower case.

```
class Person < ActiveRecord::Base
  validates_each :name, :surname do |record, attr, value|
    record.errors.add(attr, 'must start with upper case') if value =~
/\A[[:lower:]]/
  end
end
```

The block receives the record, the attribute's name and the attribute's value. You can do anything you like to check for valid data within the block. If your validation fails, you should add an error message to the model, therefore making it invalid.

3 Common Validation Options

These are common validation options:

3.1 `:allow_nil`